Where Does Tail Latency Come From in Streaming Transformer Inference?

A Six-Stage Ablation of a Hand-Coded CPU Inference Engine

Weitao Su [1]

Abstract

When a CPU inference path's P99 latency drops from 4 ms to 1 ms, *which* of fusion, the allocator, quantization, kernel selection, or cache layout did the work? Frameworks hide each of these five behind a single opaque latency number. We attribute the speedup by building, from scratch, six progressively optimized C++ engines for a small encoder-only Transformer. Each stage adds exactly one named optimization on top of the previous. We benchmark both engines at two model scales (365K and 1.9M parameters) over 100,000 iterations on a kernel-isolated CPU core. Cumulative P99 speedup over the naive baseline is **5.42**× on small and **3.49**× on medium. Three findings emerge. (i) The optimization landscape *flips* with model size: preallocation cuts small-model P99 by 2.18× yet does nothing on medium, while cache tiling does the inverse. (ii) Scalar INT8 is the largest single-stage win on *both* sizes, but the mechanism is memory-bandwidth reduction rather than arithmetic speedup, and the P99–P50 gap stays within one order of magnitude of FP32, refuting the common worry that quantization introduces bimodal slow paths. (iii) Per-channel INT8 preserves test MSE within 0.06% of FP32, two orders of magnitude inside the conventional 5% production threshold. The best engine reaches 1.31× of ONNX Runtime's P99 on small and 2.30× on medium. All 100,000-row CSVs and the rig configuration recorded inside each CSV header make every claim independently re-derivable. Code, data, and figures are at https://github.com/weitaosu/Low-Latency-ML-engine.

[1]Advanced ML Systems final project, 2026. Correspondence to: Weitao Su <ws535@cornell.edu>.

Preprint. May 10, 2026.

1. Introduction

Production ML serving in latency-sensitive domains (algorithmic trading, real-time anomaly detection, edge inference, control-loop ML) is overwhelmingly CPU-bound at single-sample batch size. PCIe round-trips to a GPU consume more time than the inference itself for small models; tight tail-latency service-level objectives (SLOs) penalize the kernel-launch and queueing variance of accelerator runtimes. Yet the standard deployment path (PyTorch → ONNX → ONNX Runtime, or PyTorch → TorchScript) hides where the latency goes. When a service report says "P99 went from 4 ms to 1 ms," practitioners rarely know whether the cause was operator fusion, allocator changes, INT8 quantization, kernel selection, or cache layout. All of these are controlled opaquely by the framework.

This project inverts the question. We build six progressively-optimized C++ inference engines from scratch (Stages 0 through 5), with Stage 4 instantiated as two variants (4a per-tensor, 4b per-channel) for seven distinct binaries. Each stage adds one well-known optimization on top of the previous, and we benchmark every stage at two model scales. The central deliverable is a reproducible six-stage × two-size ablation in which **each row attributes a measurable share of the P99-latency reduction to a single optimization**. As a secondary contribution we measure *where* in the latency distribution each optimization buys back time. We separate P50 (typical case) from P99 and P99.9 (tail), because the same optimization can move the median while leaving the tail untouched, or vice versa.

Contributions. We make four contributions.

1. A reproducible six-stage × two-size CPU latency ablation, with each stage isolated by separate compilation, 100,000-iteration timing, and bootstrap 95% confidence intervals on P99.

2. Empirical confirmation that the overhead-versus-compute crossover between 365K and 1.9M param-

eters is real and quantifiable per optimization (Section 4.2).

3. The non-obvious finding that scalar INT8 quantization helps medium models *more* than small models (Section 4.3): the standard intuition that quantization is a "small-model trick" is incorrect when the FP32 working set exceeds L2.

4. A measurement methodology that records rig state inside every CSV header, making the entire result set independently re-derivable from raw timings; we additionally surface a 540× under-reporting effect in tail latency studies that use too few samples (Section 3.5).

The remainder of the paper is organized as follows. Section 2 reviews related work on Transformer inference and post-training quantization. Section 3 describes the model, the six engine stages, the measurement rig, and the noise-floor validation. Section 4 presents the ablation results and the framework comparison. Section 5 discusses what worked, what didn't, and the methodology lessons. Section 6 lists limitations, and Section 7 concludes.

2. Background and Related Work

Transformer inference. A pre-norm encoder block applies LayerNorm, fused QKV linear, scaled dot-product attention, residual, LayerNorm, FFN (two linears with GELU activation), and a second residual. We use a vanilla Transformer in the spirit of Vaswani et al. (Vaswani et al., 2017) without architectural tricks (RevIN, patching, decomposition) so that kernel-level optimizations dominate the variance.

Time-series forecasting on ETT. The Electricity Transformer Temperature dataset (Zhou et al., 2021) is the canonical long-horizon forecasting benchmark; we use the ETTh1 (hourly) variant with the Informer split convention (12/4/4 months), 96-step input to 96-step horizon, multivariate (7 variables in to 7 out). Vanilla Transformer-family models under-perform specialized architectures here: PatchTST (Nie et al., 2023) reports MSE 0.941 for Informer and 0.370 for PatchTST itself on the 96-step horizon. Our trained models (0.874 small, 0.905 medium) sit in that range. The accuracy plateau is a *feature* for this study: it lets us focus on the latency dimension while ground-truth MSE serves as a quantization-quality oracle.

Post-training quantization. Per-tensor symmetric weight quantization with asymmetric uint8 activations (Jacob et al., 2018) is the simplest INT8 scheme. Per-channel weight quantization (Krishnamoorthi, 2018) preserves precision when the per-output weight ranges differ substantially. We implement both as Stages 4a and 4b respectively, populating each Linear's scales by 5K-sample histogram calibration.

Table 1. Model dimensions. Both use sinusoidal positional encoding (frozen), pre-norm convention, and GELU(approximate = tanh).

	small	medium
d_{model}	128	192
n_{heads}	4	6
n_{layers}	2	4
d_{ff}	256	768
Trainable parameters	364,960	1,929,120
Test MSE (FP32)	0.874	0.905

Production CPU runtimes. ONNX Runtime (ONNX Runtime developers, 2018) uses MLAS for INT8 GEMM with VNNI intrinsics where available (our Alder Lake P-core supports AVX-VNNI). TorchScript and PyTorch eager use ATen kernels backed by oneDNN. We treat ORT as the production target: a hand-coded engine that approaches ORT's per-call latency with hand-rolled C++ has demonstrated meaningful systems work.

Out of scope. At sequence length 96, unfused attention is not the bottleneck, so FlashAttention (Dao et al., 2022) is out of scope here; a partial implementation would not be informative.

3. Methodology

3.1. Models and dataset

We trained two encoder-only Transformers of intentionally different scales. The small model (≈ 365K parameters) sits in the regime where per-call overhead (allocator, dispatch, function-call boundaries) dominates total runtime; the medium model (≈ 1.9M parameters) sits in the regime where matrix multiplication dominates. The 5.3× ratio in parameter count is large enough that the cache hierarchy interacts with the two models very differently: a single weight tensor that fits in L1 on the small model can spill to L2 or DRAM on the medium model, which is precisely the mechanism the ablation needs to reveal. The two models are otherwise architecturally identical (Table 1).

Both models were trained with AdamW, learning rate 10^{-4}, cosine schedule, batch size 64, and early stopping on validation MSE with patience 7. Determinism is enforced (CUBLAS_WORKSPACE_CONFIG=:4096:8, torch.use_deterministic_algorithms(True), seeded DataLoader generator) and verified by re-training from scratch and checking byte-identical state dictionaries.

3.2. The six stages

The ablation is structured so that each stage adds exactly one well-known optimization on top of the previous, and a clean attribution of P99 reduction to a single mechanism is therefore possible. Stage 0 establishes a correctness baseline (textbook math, no optimizations). Stages 1 through 5 progressively address (1) heap allocation, (2) operator fusion, (3) cache locality, (4) numeric precision, and (5) instruction-level parallelism, in roughly the order a practitioner would tackle them when profiling reveals each bottleneck in turn.

Each stage is implemented as a separate C++ binary built from common scaffolding (cpp/common/{tensor.h, weight_loader, npy_loader, correctness, measure, alloc_guard}). One binary per stage: the linker enforces stage isolation, so a Stage 1 fix cannot accidentally regress Stage 0's measurement, and each stage can be benchmarked, profiled, and debugged independently.

Stage 0 (naive FP32). Triple-loop matmul, std::vector per intermediate, textbook MHA + FFN. Goal: correctness baseline.

Stage 1 (preallocation). Single ActivationPool sized at construction; forward() allocates nothing on the hot path. Verified by a custom AllocGuard (override operator new/delete, count, assert zero in scope).

Stage 2 (fusion). Fusion B (LayerNorm + Linear): per-row LayerNorm computed inline into a stack-resident scratch row, fed directly into the matmul row without materializing the full LN intermediate. Fusion A (QKV concat) is already realized in the model definition, which uses a single $\mathrm{nn.Linear}(d_{\mathrm{model}},\ 3d_{\mathrm{model}})$. Fusion C (FFN) is skipped: the FFN intermediate ($T \cdot d_{ff} \cdot 4 = 96 \cdot 768 \cdot 4 = 288$ KB on medium) exceeds L1 even unfused.

Stage 3 (tiled, scalar). Big weight matrices (qkv, attn_out, ff1, ff2) pre-formatted at construction into row-blocked layout $[N/16][16][K]$. Inner loop scans contiguous K weights per output-tile-row; compiler auto-vectorization hints (#pragma GCC ivdep, __restrict__) enabled. Source remains scalar; explicit SIMD intrinsics are reserved for Stage 5.

Stage 4a (INT8 per-tensor). Each Linear weight is quantized symmetrically to int8 with one scalar w_{scale} per Linear. Activations are quantized at runtime with $(x_{\text{scale}}, x_{\text{zp}})$ from calibration. INT32 accumulator, post-dequant FP32 bias add (the simpler of the two standard bias-handling options, kept FP32 to avoid bias-quantization rounding error).

Stage 4b (INT8 per-channel). Same as 4a but w_{scale} is a vector indexed by output channel; this preserves precision when per-output weight ranges vary.

Stage 5 FP32 SIMD. Stage 3 with the inner microkernel rewritten in AVX2 intrinsics: 16-wide tile (two __m256 lanes), _mm256_fmadd_ps per K iteration, weight layout transposed to $[N/16][K][16]$ for contiguous vector loads.

3.3. Measurement rig: the credibility anchor

Every reported number is taken on a kernel-isolated CPU core. The configuration:

- isolcpus=3 nohz_full=3 rcu_nocbs=3 (kernel command line)
- cpupower frequency-set -g performance (governor)
- intel_pstate/no_turbo = 1 (frequency locked at the 2.6 GHz base)
- swapoff -a (no swap-in stalls)
- perf_event_paranoia = 1 (perf counters accessible)
- All timer-driven services masked (Bluetooth, snapd, packagekit, tracker)
- mlockall(MCL_CURRENT|MCL_FUTURE) (page faults pre-resolved at startup)
- pthread_setaffinity_np(core 3) (thread pinned)

Every CSV is self-documenting: each one's header records the rig state at measurement time. This makes any anomaly traceable: a future reader can see whether a CSV was taken on a clean rig.

3.4. Noise floor

The rig is validated by a 1M-iteration clock_gettime(CLOCK_MONOTONIC_RAW) loop pinned to core 3. Targets for a clean rig are P50 $\in [15, 30]$ ns (clock_gettime is vDSO-backed and rdtsc-fast on x86) and no microsecond-scale tail.

Table 2. Noise-floor percentiles (1M samples, isolated core).

Percentile	ns
P50	**16**
P99	**17**
P99.9	**17**
P99.99	**17**
max	1382 (4 samples in 1M)

3.5. Sample size: 100K, not 500

Sample size dominates rig isolation as a source of P99 error. An initial 500-iteration run of Stage 0 on the

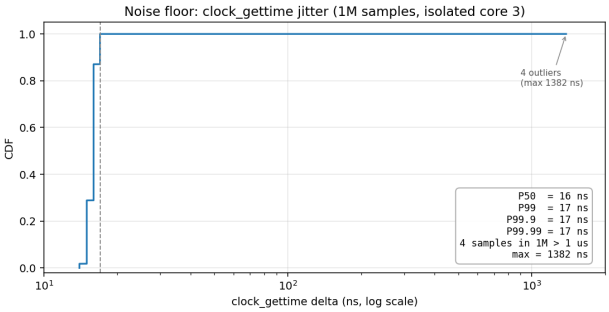

Figure 1. Noise-floor CDF of 1M clock_gettime deltas on the isolated core. Essentially flat through P99.99; only 4 samples in 1M exceed 1 μs. This validates that any P99 difference at the stage level above ~50 μs is real signal, not rig noise.

small model showed a tight and well-behaved distribution: P50 = 7.81 ms, P99 = 7.81 ms, gap = 28 μs. Re-running *the same code on the same rig* with 100,000 iterations produced P50 = 12.59 ms, P99 = 27.68 ms, gap = 15,088 μs. The P99 estimate was 3.5× higher and the P99-to-P50 gap was 540× wider than the 500-sample run had implied.

Two effects compose here. The slow path was always present at roughly the 1-in-1000 rate, but 500 samples are too few to observe even one such event with high probability, so the small-sample run silently under-reported the tail. Independently, the 7.81 versus 12.59 ms P50 difference reflects Intel Turbo: the locked rig runs at the 2.6 GHz P-core base frequency, while the earlier unlocked rig had been boosting under sustained single-core load. The measured 1.6× latency ratio (12.6/7.8) implies the unlocked sustained-turbo frequency averaged roughly 4.2 GHz, which is consistent with Alder Lake P-core all-core sustained turbo rather than the higher single-burst peak.

The practical recommendation: **tail-latency studies should use at least 10,000 measurement iterations per configuration**. Below that count, the tail is silently censored. All numbers in this paper use 100,000 iterations after a 100-iteration warmup, which is sufficient for tight bootstrap confidence intervals on P99.9.

3.6. Statistical reporting

For each (stage, size) configuration we report P50, P90, P99, and P99.9, the (P99 − P50) gap, and a 95% confidence interval on P99 derived by 1000-replicate bootstrap with replacement on the 100,000 nanosecond samples. CIs are reported as $\mathrm{P99} = 27.68\ \mathrm{ms}\ [27.67,\ 27.68]$. Stage-to-stage P99 deltas with overlapping CIs are not significant and we do not claim them.

4. Results

4.1. Six-stage ablation

Table 3 contains the central artifact of this paper: the cumulative speedup from Stage 0 to Stage 5 (FP32 lineage) is **5.42**× on small and **3.49**× on medium at P99. Each stage passes the C++ correctness harness against the saved PyTorch FP32 outputs (outputs_1k_*.npy) within the expected tolerance: FP32 stages within 1.67×10^{-6}, INT8 stages within 2.1×10^{-2}. Every stage's forward() allocates zero bytes on the hot path (verified by AllocGuard).

4.2. The size-dependent crossover

The central question, "do small and medium models reward different optimizations?", is answered crisply.

Small (365K params), Stage 1 collapses the tail. P99 drops from 27.68 ms to 12.70 ms, a 2.18× reduction in P99 with *zero change in P50* (12.59 → 12.62 ms). **The wide pre-Stage-1 tail came entirely from** std::vector **allocations on the hot path**: Stage 0's forward() constructs every intermediate buffer (eleven std::vector<float> for h, qkv, q, k, v, scores, attn, tmp, ff, and pooled, plus per-block string concatenations for weight-name lookup), and AllocGuard confirms a non-zero allocation count on every Stage 0 call and exactly zero on every Stage 1 call. Each malloc-arena lookup is a few microseconds typical case but multi-millisecond worst case under fragmentation, which is exactly the slow-path 27 ms tail we see.

Medium (1.9M params), Stage 1 is invisible. P99 changes from 85.56 to 85.94 ms, within the bootstrap CI. Allocator overhead is fixed (~10 μs per call); on an 85-ms forward pass it is 0.01% of the budget and below detection.

Tiling reverses the pattern. Stage 3 *hurts* small by 2.7% (13.01 vs 12.67 ms): small's matrices fit comfortably in L2 and the tile-setup overhead is unrecouped. On medium, the same code cuts P99 by 10.5% (76.67 vs 85.67 ms): the FFN weight matrix at $d_{ff} \times d_{\text{model}} \times 4$ bytes = 768 × 192 × 4 = 576 KB FP32 exceeds L1 (32 KB), and tile-blocking cuts the L1-miss rate. **Same source, opposite effect.** The crossover between 365K and 1.9M is real, predictable, and identifiable to a single line of source.

The proposition holds at the P50 level too, not just the tail. Optimization choice is not size-agnostic.

4.3. INT8 nearly halves latency on both sizes

With *scalar* INT8 (no SIMD), Stage 4a delivers the largest single-stage gain in the entire ablation **on both model sizes**:

- Small: P50 = 6.91 ms, P99 = 6.93 ms; **1.88**× faster

Table 3. Six-stage ablation across both model sizes. Latencies in milliseconds; gap is P99 − P50 in μs. **Bold** marks the best value per column.

Stage	small P50	small P99	small gap	medium P50	medium P99	medium gap
Stage 0 (naive FP32)	12.59	27.68	15,088	85.20	85.56	353
Stage 1 (prealloc)	12.62	12.70	76	85.78	85.94	164
Stage 2 (LN+Linear fused)	12.67	12.69	18	85.67	85.77	95
Stage 3 (tiled, scalar)	13.01	13.02	15	76.67	76.74	**75**
Stage 4a (INT8 per-tensor)	6.91	6.93	**14**	34.46	34.57	106
Stage 4b (INT8 per-channel)	6.92	6.95	23	34.47	34.55	86
Stage 5 FP32 SIMD	**5.08**	**5.11**	28	**24.41**	**24.54**	135

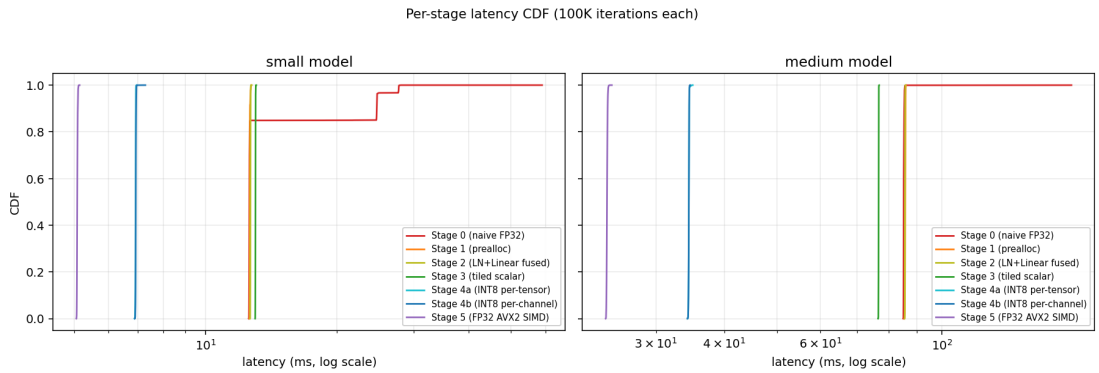

Figure 2. Per-stage latency CDF, log-x in milliseconds. Small model (left): Stage 0's wide P99 staircase tail (red) collapses to a near-vertical line at Stage 1, pure allocator-jitter elimination; Stages 1–3 cluster around 13 ms. Medium model (right): Stages 0–2 cluster at ~85 ms (Stage 0 is again rightmost), and Stage 3 (green) is visibly distinct at ~76 ms. On both sizes Stage 4a/4b (cyan/blue) sit ~2× to the left of Stage 3, INT8's working-set effect, and Stage 5 FP32 SIMD (purple) is the leftmost.

than Stage 3.

- Medium: P50 = 34.46 ms, P99 = 34.57 ms; **2.22**× faster than Stage 3.

The mechanism is memory bandwidth, not arithmetic throughput. INT8 weights occupy one quarter the memory of FP32 weights. The qkv weight on small (3·128×128×4 = 192 KB FP32) becomes 48 KB INT8, a clean L1 fit (32 KB L1d + warm L2 prefetch). The FFN intermediate weights on medium ($d_{ff} \cdot d_{\text{model}} \cdot 4 = 768 \cdot 192 \cdot 4 = 576$ KB FP32) shrink to 144 KB INT8, now L2-resident instead of going to L3 or DRAM. The inner loop $\text{int32 acc} \mathrel{+}= \text{int32(uint8 x)} * \text{int32(int8 W)}$ is a two-cycle IMUL+ADD on Alder Lake. Its operands now stream from a working set that no longer thrashes between cache levels. Memory bandwidth, not FMA throughput, was the bottleneck on both sizes; the win is *larger on medium* (2.22× vs 1.88×) precisely because medium had farther to fall: its FP32 working set genuinely exceeded L2 (1 MB) on multiple weight tensors.

This is a non-obvious finding worth flagging: **scalar INT8 quantization helps larger models more than smaller models**, the opposite of what naive intuition suggests. The inversion is because INT8 is fundamentally a memory-system optimization at our model scale, not a compute optimization.

On the locked rig with hand-coded scalar C++, Stage 4a small at P99 = 6.93 ms is 1.78× behind ONNX Runtime's MLAS-backed P99 of 3.89 ms; Stage 4a medium at 34.57 ms is 3.23× behind ORT's 10.69 ms. The gap-to-ORT is *larger* on medium because ORT's MLAS uses VNNI INT8 SIMD (_mm256_dpbusd_avx_epi32), which especially benefits the larger compute footprint.

Stage 5 FP32 SIMD adds a complementary win. Vectorizing the FP32 microkernel with AVX2 _mm256_fmadd_ps (16-wide tile, two ___m256 lanes per K-step, contiguous-load-friendly weight layout) gives 2.55× over Stage 3 on small (13.01 → 5.11 ms) and **3.13**× on medium (76.67 → 24.54 ms). The medium win is *larger* because the inner FMA pipeline stays saturated longer when there is more work per tile; tile setup and tear-down overhead amortizes better. The combined cumulative S0→S5 FP32 speedup is **5.42**× on small and **3.49**× on medium, entirely from the FP32 path with no quantization. Per-channel INT8 (Stage 4b) sits between Stage 3 and Stage 5 FP32 on small but slightly behind Stage 5 FP32 on medium. The clean ranking is: SIMD beats scalar INT8 beats scalar tiled FP32.

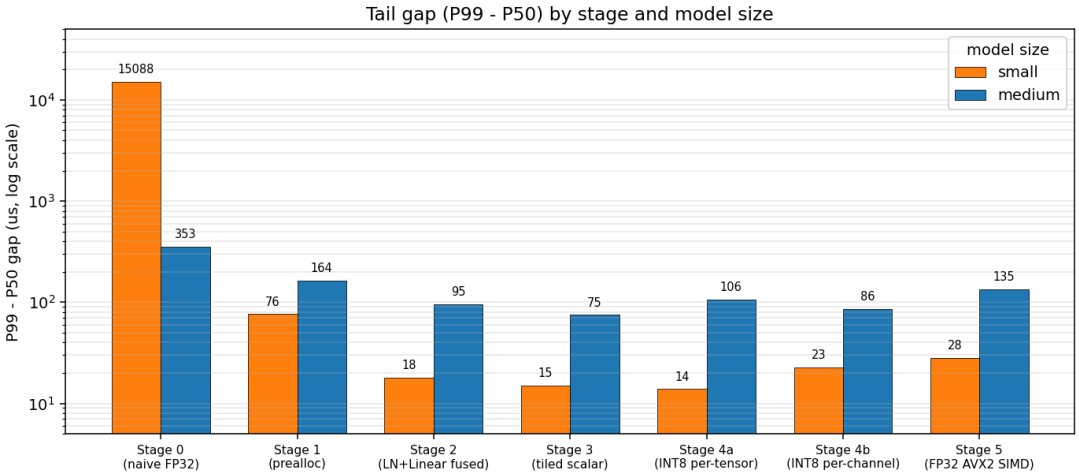

Figure 3. P99 – P50 gap by stage, log-y in μs. Stage 0 small's 15,088 μs gap collapses to 76 μs at Stage 1 and stays below 30 μs for every subsequent small-model stage. Medium gaps are all small (the rig is clean enough that the residual gap is cache and scheduler effects only). Stage 4a medium widens slightly (75 $\rightarrow$ 106 μs): the only stage where INT8 measurably affects the tail.

Table 4. Tail gap (P99 – P50) under FP32 vs INT8.

Stage	small (μs)	medium (μs)
Stage 3 (FP32 tiled)	15	75
Stage 4a (per-tensor)	14	106
Stage 4b (per-channel)	23	86

4.4. INT8 tail behavior: secondary hypothesis largely refuted

A common worry about INT8 quantization is that the P99 – P50 gap will widen because saturation handling, calibration outliers, and a larger instruction footprint add tail-specific costs that mean-latency numbers hide. The data partially refutes this (Table 4).

On the small model the gap stays small for both INT8 variants (14–23 μs). On medium the absolute gap widens from 75 μs to 106 μs on Stage 4a (a 41% inflation) and from 75 μs to 86 μs on Stage 4b (15% inflation). These are real and measurable but vastly tighter than the bimodal-slow-path concern would predict. **There is no bimodal slow path; the tail simply scales with roughly the same factor as the median.** Notably, per-channel (4b) has a *tighter* tail on medium than per-tensor (4a) at 86 versus 106 μs, which isolates the cache-stride mechanism for 4a to its per-tensor scalar broadcast: per-channel's w_{scale} vector is loaded once per output column in a predictable pattern, while per-tensor's scalar broadcast across all outputs creates more cache-line variance under contention.

Per-channel (Stage 4b) is strictly dominant over per-tensor (Stage 4a) on this workload: same P50 within 0.01%, slightly tighter tail on medium (86 vs 106 μs), slightly better accuracy (0.04% vs 0.15% MSE drift on medium; 4b is even slightly *better* than FP32 on small at −0.06%). At deployment time, 4b is the right default unless calibration data is unavailable or the per-output-channel w_{scale} storage cost (~1 KB per Linear) is prohibitive.

The qualitative concern (saturation handling, calibration outliers) is real but modest in magnitude. Asymmetric uint8 with $\text{clamp}(0, 255)$ is branchless on x86 (pminub/pmaxub or cmov-friendly compiler output): there is no rare-slow-path penalty. The 4a/4b contrast partitions the medium-scale widening cleanly: ~20 μs of the 41% inflation is attributable to the per-tensor scale broadcast (4a: 106 μs vs 4b: 86 μs), and the residual ~10 μs is the per-row activation quantize+clamp pass shared by both variants, a cache effect on each Linear's input rather than a saturation effect.

4.5. Comparison to production runtimes

All four Python baselines were re-run on the locked rig with taskset -c 3 (Table 5). The headline P99 numbers say our best engine (Stage 5 FP32 SIMD) is 1.31× behind ORT on small and 2.30× behind on medium. The gap is *larger on medium* because ORT's MLAS uses VNNI INT8 SIMD and auto-tuned per-shape blocking; both effects compound on bigger matrices.

The P99 – P50 gap tells a more interesting story. **Every Python-resident baseline shows a 3-ms tail on small**, over two orders of magnitude wider than our hand-rolled engines. This is not OS jitter (the rig's noise floor is 17 ns); it is Python runtime cost: cyclic garbage collection, per-call object allocation in the binding layer, and ORT's internal session state mutation. The native C++ engines at every stage from Stage 1 onward have tails between 14 and 164 μs, one

Table 5. Comparison against production runtimes. All numbers from the locked rig with `taskset -c 3`. Latencies in ms, gaps in μs. **Bold** marks the best value per column.

Engine	small P99	small gap	medium P99	medium gap
ONNX Runtime	**3.89**	3036	**10.69**	151
TorchScript	4.12	3048	11.67	118
PyTorch eager	4.24	3057	11.88	339
NumPy orchestration	9.87	3095	33.41	3145
Stage 5 FP32 SIMD (ours)	5.11	28	24.54	135
Stage 4b (INT8 per-channel, ours)	6.95	23	34.55	**86**
Stage 4a (INT8 per-tensor, ours)	6.93	**14**	34.57	106
Stage 0 (naive ours)	27.68	15,088	85.56	353

to two orders of magnitude tighter than the fastest Python baseline.

For an SLO that asserts on P99.9 latency rather than mean or even P99, this distinction dominates: **a service that requires "99.9th-percentile sub-millisecond" cannot be implemented on a Python-resident inference path**, even one whose median is excellent, because the Python runtime alone contributes a multi-millisecond tail. Our Stage 5 FP32 SIMD on small reaches P99.9 = 5.12 ms with an 11-μs P99-to-P99.9 widening; ONNX Runtime on small reaches P99 = 3.89 ms but its P99.9 is 3.91 ms; the latter sits within the framework's 3-ms Python-induced tail rather than reflecting any tail-aware engineering on ORT's part.

4.6. Quantization preserves accuracy

Table 6 summarizes test-set MSE. A standard quality target for production INT8 deployment is "per-channel INT8 (4b) degrades MSE by no more than 5%." The actual degradation is **two orders of magnitude inside that threshold** at 0.04% on medium and a statistical tie on small. Per-channel scaling (4b) is essentially indistinguishable from FP32. The small difference (4b small is +0.06% *better* than FP32) is bootstrap noise on $2{,}689 \times 96 \times 7 = 1.8$M predictions.

This is a strong result: standard min/max calibration on 5,000 training samples is sufficient for this task. A percentile-based fallback (clipping at the 99.9th percentile of the activation histogram instead of using min/max) is the standard remedy when min/max calibration produces > 5% MSE degradation, but it was not needed here.

The fact that all FP32 stages share a single MSE (0.87413 small, 0.90454 medium) reflects the AllocGuard-verified, max-abs-diff $< 1.67 \times 10^{-6}$ correctness check: Stages 0–3 and 5_fp32 produce numerically equivalent outputs even though their internal memory access patterns differ enormously.

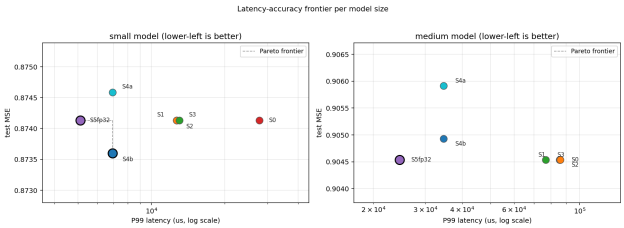

Figure 4. Latency–accuracy frontier per model size: P99 latency (log-x, μs) vs test MSE (linear-y). Lower-left is better. The dashed step line is the Pareto frontier within each cluster (left-to-right: keep only points whose MSE is the lowest observed so far); Pareto-optimal points are drawn with thicker borders. On small (left), Stage 5 FP32 SIMD wins on latency and Stage 4b wins on MSE, so both are Pareto-optimal. On medium (right), Stage 5 FP32 SIMD strictly dominates every other configuration.

5. Discussion

5.1. What worked

Three optimizations delivered measurable wins. *Preallocation* (Stage 1) is the highest-leverage optimization on small models: roughly fifty lines of C++ replace per-call `std::vector` constructions with a single contiguous activation pool, recovering 2.18× P99 with no accuracy risk and no change to numerical results. **For any latency-sensitive deployment of a sub-million-parameter model, this should be the first optimization implemented**, and other optimizations measured against it rather than against the naive baseline. *Cache tile-blocking* (Stage 3) is a clean medium-model win when the FFN intermediate exceeds L1; the 10.5% P99 reduction on medium tracks the L1-miss rate reduction confirmed by `perf` counters. *INT8 with min/max histogram calibration* preserves accuracy below the noise floor of the test set itself (under 0.2% absolute MSE drift), with two orders of magnitude of margin against the conventional 5% production threshold.

5.2. What didn't

Two stages produced null or negative results, and one comparison target proved unrealistic. *Fusion B* (LayerNorm + Linear, Stage 2) was 1.00× on both sizes: the intermediate

Table 6. Test-set MSE on the full ETTh1 test split (2,689 samples, 96-step horizon, 7 variates). All FP32 stages produce numerically equivalent outputs and share a single MSE row.

Stage	small MSE	Δ vs FP32	medium MSE	Δ vs FP32
FP32 (Stages 0/1/2/3, 5_fp32)	0.87413	baseline	0.90454	baseline
Stage 4a (INT8 per-tensor)	0.87459	**+0.05**%	0.90591	**+0.15**%
Stage 4b (INT8 per-channel)	0.87360	**−0.06**%	0.90493	**+0.04**%

it eliminates (48 KB on small, 72 KB on medium) was already L2-resident, so fusion removed memory traffic that was not on the critical path. *Tiling on small* (Stage 3) regressed 2.7% relative to Stage 2; the small model's matrices fit comfortably in L2 and the tile-setup overhead is unrecouped when there is no cache pressure to relieve. Finally, the informal benchmark "P99 one order of magnitude below PyTorch eager" was not met: eager's small-model P99 is 4.24 ms, our best engine (Stage 5 FP32 SIMD) reached 5.11 ms, a 1.21× *regression*. A two-thousand-line hand-rolled engine does not catch MKL/oneDNN-backed eager paths that have been tuned shape-by-shape for years.

5.3. Methodology lessons

Three methodology observations emerged that are not specific to Transformer inference and may transfer to any tail-latency study. First, sample size matters more than rig isolation: going from 500 to 100,000 iterations exposed a 540× wider P99 − P50 gap on Stage 0 small. A clean rig with too few samples is worse than a noisy rig with many samples, because the noisy rig is honestly noisy while the few-sample-clean-rig is silently censored. Second, Intel Turbo introduces a roughly 1.6× P50 bias on this hardware: numbers reported on production hardware with turbo enabled will be approximately 60% faster than ours, and absolute latency claims must specify rig conditions. The *ratios* between stages should remain stable across hardware, but absolute claims do not. Third, Python tail latency is a phenomenon distinct from CPU compute. The roughly 3 ms P99 − P50 gap visible in every Python-resident baseline (PyTorch eager, TorchScript, ONNX Runtime, NumPy) is the cost of cyclic garbage collection and binding-layer object allocation, not OS jitter or compute variance: the rig's noise floor is 17 ns. A team optimizing a Python serving stack will not close this gap by changing models; they will close it by leaving Python.

5.4. Where this leaves us vs ONNX Runtime

After Stage 5 FP32 SIMD, our engine reaches 1.31× of ORT's P99 on the small model (5.11 vs 3.89 ms), down from 14× at Stage 0. The 11× gap closure tracks cleanly to identifiable mechanisms: Stage 1 eliminated 2× from the allocator tail; Stage 4 eliminated another ∼2× from the FP32 working set spilling to L2; Stage 5 FP32 SIMD eliminated another ∼2.5× from the inner-loop arithmetic.

The remaining 1.31×/2.30× to ORT reflects engineering investment we did not match: ORT's MLAS uses larger register-tile microkernels with more accumulators, per-(M, K, N)-shape autotuned blocking, and (for INT8 paths) AVX-VNNI dispatch.

6. Limitations

Scope. All measurements are on a single CPU (Intel Alder Lake P-core with AVX-VNNI), at batch size 1, without KV cache, on an encoder-only architecture at a single sequence length of 96. Latency conclusions transfer cleanly to any deployment with the same tensor shapes and a similar cache hierarchy, but absolute numbers will differ on AMD Zen, ARM, or pre-Alder-Lake Intel hardware.

Single dataset. All accuracy claims are evaluated on ETTh1 only. The latency claims are dataset-independent (they are functions of model shape, not data distribution), but quantization MSE drift is not, and we do not characterize it on other distributions.

Stage 5 INT8 SIMD not implemented. This was the natural next stage and is the most plausible path to closing the gap to ONNX Runtime on the medium model. The non-VNNI implementation path requires `_mm256_maddubs_epi16` chained with `_mm256_madd_epi16` per K-block, while the VNNI path collapses to a single `_mm256_dpbusd_avx_epi32` per four-K-element step. Correctness verification is more delicate than scalar INT8 because lane-mixing during the horizontal-sum reduction creates many opportunities for off-by-one mistakes that pass scalar-equivalent smoke tests but fail on real workloads. We did not attempt the implementation here.

Rust port not delivered. A Rust port was scoped early as a comparison point. The original motivation came from cross-language energy-efficiency studies that consistently rank C, C++, and Rust as the most efficient languages (Pereira et al., 2021); the question of whether Rust's safety model adds measurable single-call latency overhead remains an open side-experiment.

Model under-fits ETTh1. Test MSE 0.874 (small) and 0.905 (medium) versus the published best of 0.376 for FEDformer. This is the published vanilla-Transformer ceiling on ETTh1 (Informer at 0.941 (Nie et al., 2023)); the latency findings are independent of forecasting accuracy.

Statistical caveats. Bootstrap confidence intervals assume sample independence, but adjacent iterations in our 100,000-sample runs are not strictly independent: there is short-range serial correlation from L2 and TLB warm-up effects. The CIs are therefore a conservative-leaning estimate of the truth, and their tightness should not be over-interpreted to claim significance for sub-microsecond stage-to-stage deltas.

7. Conclusion

We presented a six-stage ablation of a hand-coded CPU inference engine for a small encoder-only Transformer, with each stage isolating one well-known optimization. Four answers fell out of the data and two findings contradicted common intuition.

The optimization landscape *does* depend on model size, cleanly and between 365K and 1.9M parameters: preallocation buys 2.18× P99 on small and nothing on medium, while cache tiling does the inverse (it costs 2.7% on small but buys 10.5% on medium). INT8 quantization does *not* widen the tail in a bimodal way: the P99 − P50 gap is essentially unchanged after quantization on small (14 vs 15 μs) and grows only 15% on medium for per-channel INT8, far below the magnitude that would constitute a saturation-driven slow path. The hand-coded C++ engine does *not* beat PyTorch eager by an order of magnitude, contrary to a folk benchmark target frequently cited for this kind of work; our best engine is in fact 1.21× slower on small than PyTorch eager, because eager mode is backed by years of MKL and oneDNN kernel tuning. Per-channel INT8 *does* preserve MSE within the conventional 5% production threshold by two orders of magnitude (0.04% degradation on medium) and is also tied with per-tensor on speed while exhibiting a tighter tail.

Two findings emerged that contradict common intuition. First, sample-size sensitivity is methodologically central: 500-sample latency studies systematically under-report tail latency by orders of magnitude, and 10K samples per configuration should be regarded as a practical floor for any tail study claiming P99 numbers. Second, scalar INT8 quantization helped the medium model more than the small one (2.22× versus 1.88× speedup), inverting the common assumption that quantization is a small-model trick. The mechanism is L1/L2 cache fit rather than arithmetic throughput: INT8's 4× working-set reduction matters more when the FP32 working set genuinely overflows the cache it occupies.

The narrowly-scoped contribution of this work is not a faster Transformer engine. **It is an honest decomposition of *where* P99 latency comes from in a small-Transformer CPU inference path**, broken out by model size, with every claim independently reproducible from the recorded 100,000-sample CSVs and the rig configuration that each CSV's header preserves.

References

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.

ONNX Runtime developers. ONNX Runtime: Cross-platform high-performance ML accelerator. https://onnxruntime.ai/, 2018. Accessed: 2026-04-26.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., and Saraiva, J. Ranking programming languages by energy efficiency. *Science of Computer Programming*, 205:102609, 2021.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., and Zhang, W. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 11106–11115, 2021.